

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М41-мм

Отделение некоррелированных  
подзапросов как метод оптимизации  
запросов в колоночных СУБД

*Кузин Яков Сергеевич*

Отчёт по преддипломной практике  
в форме «Решение»

Научный руководитель:  
ассистент кафедры информационно-аналитических систем Чернышев Г. А.

Санкт-Петербург  
2026

# Оглавление

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Введение</b>                                             | <b>3</b>  |
| <b>1. Постановка задачи</b>                                 | <b>5</b>  |
| <b>2. Обзор</b>                                             | <b>6</b>  |
| 2.1. Коррелированные подзапросы . . . . .                   | 6         |
| 2.2. Существующие методы оптимизации . . . . .              | 8         |
| <b>3. Предложенный метод</b>                                | <b>9</b>  |
| 3.1. Классы запросов . . . . .                              | 10        |
| 3.2. Преобразование запроса в эквивалентный . . . . .       | 11        |
| 3.3. Сценарии применения . . . . .                          | 19        |
| <b>4. Реализация</b>                                        | <b>21</b> |
| 4.1. Архитектура PosDB . . . . .                            | 21        |
| 4.2. LP оператор . . . . .                                  | 22        |
| 4.3. Реализация недостающих операторов . . . . .            | 25        |
| <b>5. Эксперименты</b>                                      | <b>27</b> |
| 5.1. Условия экспериментов . . . . .                        | 27        |
| 5.2. Классы запросов, исключенные из рассмотрения . . . . . | 27        |
| 5.3. Проведенные эксперименты . . . . .                     | 29        |
| <b>Заключение</b>                                           | <b>38</b> |
| <b>Список литературы</b>                                    | <b>39</b> |

# Введение

Колоночные системы управления базами данных (СУБД) организуют физическое хранение данных по столбцам, что не делает их универсально оптимальным решением, но в ряде сценариев позволяет добиться высокой эффективности [2]. В частности, при выполнении аналитических запросов подобная организация данных часто превосходит классическую модель. Когда запрос требует сканирования больших объемов данных, но затрагивает лишь ограниченный набор атрибутов, доступ к отдельным столбцам сокращает избыточные операции ввода-вывода. В результате время выполнения таких запросов уменьшается, поскольку подобные операции остаются одними из наиболее ресурсоемких этапов обработки данных [25].

С точки зрения пользователя колоночные системы не отличаются от реляционных: запросы пишутся на привычном языке SQL, а результаты выдаются в стандартной табличной форме. В отличие от этого, реализация таких систем существенно отличается и нередко сопряжена не только с техническими трудностями, но и с различными особенностями [11]. Например, это могут быть особые требования к хранению и обработке данных, архитектурные компромиссы, а также иной подход к проектированию системы в целом.

Подзапросы представляют собой одну из ключевых частей любого хоть сколько-нибудь сложного запроса и широко используются как рядовыми пользователями, так и экспертами. Так, например, в эталонном тестовом наборе ТРС-Н [23] более половины запросов включают один или несколько подзапросов, что подчеркивает необходимость их полноценной поддержки в любой СУБД.

Эффективность обработки запросов определяется выбранным планом выполнения [8]. Выделяются две взаимосвязанные стадии этого процесса: генерация и выбор плана запроса, а также его непосредственное исполнение. Отсутствие специализированных механизмов на любой из этих стадий может сделать СУБД непригодной для промышленного применения, где предъявляются строгие требования как к скорости

обработки данных, так и к использованию ресурсов.

В настоящей работе основное внимание сосредоточено на второй стадии: планировщику передаются операторы (фрагменты плана) и обеспечивается их эффективное исполнение. В этом контексте рассмотрен новый метод обработки запросов, содержащих составные коррелированные подзапросы. Его основная идея заключается в отделении из таких подзапросов некоррелированных частей. Это позволяет получить многократный прирост производительности, если выделенная часть позволяет исключить из обработки значительную часть данных.

Для исследования подобных случаев было определено множество классов запросов. Среди них выделены те, на которых применение рассматриваемого метода обеспечивает существенное увеличение производительности. Кроме того, был произведен анализ эффективности предложенного подхода в зависимости от селективности некоррелированных предикатов, а также выявлены преимущества его использования в позиционной (position-enabled) системе управления базами данных.

Примером такой системы может служить PosDB [19]. Это колоночная СУБД, поддерживающая позиционное представление данных [9] и позднюю материализацию [1, 13]. В ее архитектуру заложена возможность доступа к данным через позиции — положения записей внутри колонок. Это позволяет сократить объем передаваемых между операторами данных, что повышает эффективность всей системы и открывает широкие возможности для различных оптимизаций. Совокупность этих свойств делает PosDB уникальной среди известных решений. Ранее в ней отсутствовали механизмы оптимизации подзапросов, что и послужило отправной точкой для их внедрения.

Для проверки работоспособности предложенного подхода были проведены эксперименты с использованием PostgreSQL и PosDB в вариантах с рассматриваемыми оптимизациями и без них. Результаты показали, что при определенных условиях применение этого метода может обеспечить многократное увеличение производительности.

# 1. Постановка задачи

Целью настоящей работы является разработка метода оптимизации выполнения запросов в колоночной позиционной СУБД, основанного на выделении из составных подзапросов некоррелированных частей. Для ее достижения были поставлены следующие задачи:

1. Ознакомиться с предметной областью обработки коррелированных подзапросов. По результатам ознакомления написать обзор существующих методов оптимизации их выполнения.
2. Выделить классы запросов, на которых может быть применен метод оптимизации выполнения подзапросов, основанный на выделении из них некоррелированных частей. Определить те из них, где его применение обеспечивает увеличение производительности.
3. Разработать метод оптимизации выполнения подзапросов и внедрить его в архитектуру позиционной колоночной СУБД PosDB.
4. Реализовать недостающие операторы для рассматриваемой СУБД, чтобы обеспечить возможность проведения экспериментов на всех выделенных классах запросов.
5. Осуществить сравнительный анализ производительности системы до и после внедрения предложенного метода оптимизации.

## 2. Обзор

Реляционные СУБД являются основой для организации, хранения и обработки данных [21]. Одним из их ключевых средств, позволяющих решать сложные задачи, а также повышать гибкость работы с данными, являются подзапросы — запросы, находящиеся внутри других запросов. Например, в эталонном тестовом наборе ТРС-Н [23] свыше половины запросов включают один или несколько подзапросов.

Подзапросы обладают как достоинствами, так и недостатками [3, 10]:

- + Повышение читаемости и модульности SQL-кода.
- + Облегчение сопровождения этого кода.
- + Сокращение числа соединений.
- Возможное увеличение времени выполнения запросов.
- Уменьшение ясности основного запроса.
- Формирование менее переиспользуемых планов, что снижает эффективность кеширования.

Возможное увеличение времени выполнения запросов особенно выражено для коррелированных подзапросов, поскольку внутренний запрос выполняется заново для каждой строки внешнего. Поэтому оптимизация подзапросов остается важной практической задачей, решение которой может принести заметную пользу в рассматриваемой области баз данных.

### 2.1. Коррелированные подзапросы

Коррелированные подзапросы представляют собой вложенные запросы, которые ссылаются на значения столбцов внешнего запроса. Пример представлен в листинге 1. С его помощью можно найти все

книги, для которых существуют писатели со следующими характеристиками: прозвище совпадает с именем автора книги, рейтинг выше 7, возраст не более 25 лет.

### Листинг 1: Пример коррелированного подзапроса с EXISTS

```
SELECT B.title
FROM books AS B
WHERE EXISTS (
    SELECT *
    FROM writers AS W
    WHERE W.nickname = B.author_name AND
        W.age <= 25 AND
        W.rating > 7
);
```

Подобные подзапросы могут находиться после любого оператора сравнения с множеством. Например, в листинге 2 показан запрос с ALL. С его помощью можно найти все книги, имеющие рейтинг, который больше соответствующих значений среди всех книг, которые либо имеют больше 550 страниц, либо не являются любимыми.

### Листинг 2: Пример коррелированного подзапроса с ALL

```
SELECT B.title
FROM books AS B
WHERE B.rating >= ALL (
    SELECT L.rating
    FROM liked_books AS L
    WHERE L.pages > 550 OR
        L.book_id != B.id
);
```

Узкое место подобных запросов — необходимость в выполнении вложенного запроса для каждой строки внешнего. Без различных техник оптимизации их выполнение на достаточно больших данных может оказаться неприемлемо долгим.

## 2.2. Существующие методы оптимизации

Формально подзапросы не входят в классическую реляционную алгебру, поскольку та оперирует множествами данных, а не иерархическими вложениями [6]. Чтобы восполнить этот пробел, предложены различные расширения этой алгебры, а также техники, сводящие подзапросы к эквивалентным выражениям с помощью соединений и других операций. Так, авторы учебника [21] демонстрируют, что конструкции с EXISTS и NOT EXISTS можно заменить комбинациями полусоединения и антисоединения, а в работе [18] показано, как подзапросы со сравнением с множеством (например,  $< \text{SOME}$  или  $> \text{ALL}$ ) преобразуются с использованием специальных соединений с маркировкой.

Предложенный подход к обработке составных предикатов расширяет существующие методы оптимизации подзапросов. В частности, это касается исследований из работ [7] и [8]. Они посвящены объединению, устранению и декорреляции подзапросов. В отличие от них, предлагаемая методика акцентирует внимание на выделении некоррелированной части составных предикатов с целью снижения накладных расходов. То есть она представляет собой подход, применимый в случаях, когда полная декорреляция нецелесообразна.

Существует и ряд концептуально других методов. Например, в работе [4] обсуждается фреймворк для полной декорреляции в Microsoft Fabric, а в статье [17] рассмотрено ресурсоемкое выполнение запросов. Кроме того, существует метод QuerySplit [24], который работает на уровне логического плана. Однако все эти методы требуют значительные затраты ресурсов на их внедрение.

Основа предлагаемой оптимизации — работа с предикатами. Она уменьшает вычислительные издержки при выполнении запросов и при этом не требует существенной переработки как оптимизатора, так и механизмов их исполнения.



### 3. Предложенный метод

В работе представлен новый подход к обработке подзапросов для случаев, когда в блоке WHERE встречаются составные предикаты с коррелированной и некоррелированной составляющими. Его ключевая идея заключается в отделении некоррелированной части и ее последующем применении для уменьшения объема вычислений.

В реляционных СУБД подзапросы, как правило, содержат логические операторы (OR, AND). Их использование может значительно усложнить выполнение запроса, особенно когда речь идет о больших объемах данных [16]. В листинге 3 показан пример запроса с подобными связками.

**Листинг 3: Подзапрос с логическими связками**

```
SELECT S.name
FROM students AS S
WHERE S.name IN (
    SELECT T.name
    FROM teachers AS T
    WHERE T.subject = 'Math' OR
           T.salary > 50000 AND
           T.age < 30
);
```

Возможная стратегия оптимизации заключается в разложении сложных подзапросов на ряд более простых, используя специальные правила. Полезно отделять некоррелированные части от коррелированных: первые можно вычислить заранее, что сокращает число повторных вычислений при обработке оставшейся части и, как следствие, повышает эффективность выполнения запроса в целом.

Похожий прием применяется в компиляторах различных языков программирования [15]: в выражениях вида “X AND Y AND Z” вычисление прекращается при первом False, а в “X OR Y OR Z” — при первом True. В методике, рассматриваемой в данной работе, вычисления также могут прерываться досрочно: как только некоррелированная часть однозначно определяет итог.

### 3.1. Классы запросов

Для оценки применимости предложенного метода выделены классы запросов, отраженные в списке ниже. Под С подразумевается коррелированный предикат, а под N — некоррелированный.

1. SELECT ... WHERE X IN (SELECT ... WHERE N AND C)
2. SELECT ... WHERE X IN (SELECT ... WHERE N OR C)
3. SELECT ... WHERE EXISTS (SELECT ... WHERE N AND C)
4. SELECT ... WHERE EXISTS (SELECT ... WHERE N OR C)
5. SELECT ... WHERE X = SOME (SELECT ... WHERE N AND C)
6. SELECT ... WHERE X = SOME (SELECT ... WHERE N OR C)
7. SELECT ... WHERE X <> SOME (SELECT ... WHERE N AND C)
8. SELECT ... WHERE X <> SOME (SELECT ... WHERE N OR C)
9. SELECT ... WHERE X < SOME (SELECT ... WHERE N AND C)
10. SELECT ... WHERE X < SOME (SELECT ... WHERE N OR C)
11. SELECT ... WHERE X > SOME (SELECT ... WHERE N AND C)
12. SELECT ... WHERE X > SOME (SELECT ... WHERE N OR C)
13. SELECT ... WHERE X = ALL (SELECT ... WHERE N AND C)
14. SELECT ... WHERE X = ALL (SELECT ... WHERE N OR C)
15. SELECT ... WHERE X <> ALL (SELECT ... WHERE N AND C)
16. SELECT ... WHERE X <> ALL (SELECT ... WHERE N OR C)
17. SELECT ... WHERE X < ALL (SELECT ... WHERE N AND C)
18. SELECT ... WHERE X < ALL (SELECT ... WHERE N OR C)

19. SELECT ... WHERE X > ALL (SELECT ... WHERE N AND C)

20. SELECT ... WHERE X > ALL (SELECT ... WHERE N OR C)

Каждый из запросов может содержать отрицание перед оператором сравнения с множеством. В связи с этим появляется еще 20 аналогичных классов, включающих оператор NOT. Все перечисленные классы необходимо рассматривать отдельно, поскольку для многих из них требуются специфические преобразования, а некоторые допускают сразу несколько вариантов трансформаций.

## 3.2. Преобразование запроса в эквивалентный

Ключевым этапом предлагаемого в настоящей работе метода является приведение запроса к эквивалентной форме. Конкретная трансформация зависит от класса запроса: в отдельных случаях требуется дублирование некоррелированного предиката или замена оператора сравнения с множеством на противоположный. Далее детально рассматриваются все используемые преобразования.

### 3.2.1. Стандартное преобразование

Начнем анализ предложенной оптимизации с запроса класса 10 из листинга 4. Самый простой способ выполнить этот запрос — для каждой строки таблицы part проходить по таблице lineitem и проверять соответствующий предикат. Однако данный подход требует большого количества вычислений, многие из которых избыточны.

#### Листинг 4: Исходный запрос

```
SELECT P.partkey
FROM part AS P
WHERE P.retailprice < SOME (
    SELECT 3 * L.extendedprice * L.discount * L.tax
    FROM lineitem AS L
    WHERE L.supkey = 451 OR P.size = L.quantity
);
```

Суть предложенного метода — выделить некоррелированную составляющую и применить ее как фильтр, чтобы уменьшить число проходов по внутренней таблице. Для этого запрос преобразуется, как показано в листинге 5.

### Листинг 5: Преобразованный запрос

```
SELECT P.partkey
FROM part AS P
WHERE P.retailprice < SOME (
    SELECT 3 * L.extendedprice * L.discount * L.tax
    FROM lineitem AS L
    WHERE L.suppkey = 451
) OR P.retailprice < SOME (
    SELECT 3 * L.extendedprice * L.discount * L.tax
    FROM lineitem AS L
    WHERE P.size = L.quantity
);
```

Получившийся запрос включает два подзапроса в WHERE. Первый — некоррелированный. Его результат не зависит от таблицы part и потому может быть вычислен за один проход по lineitem. Вторым — коррелированный, выполняющийся отдельно для каждой строки part и требующий полного обхода lineitem. В результате для каждой записи из part сперва выполняется “дешевый” некоррелированный подзапрос, и только при ложном его результате запускается более затратный коррелированный.

Преобразование запросов из других классов происходит схожим образом. Однако оно может потребовать дублирования некоррелированного предиката или замены оператора сравнения с множеством на противоположный. Все эти преобразования представлены в таблицах 1 и 2. В них \* означает, что использование соответствующего правила возможно только при условии, что  $N \cap C \neq \emptyset$ . Здесь под N и C подразумеваются множества, образованные значениями, которые удовлетворяют некоррелированному и коррелированному предикату соответственно.

Таблица 1: Преобразования запросов с операторами IN, EXISTS, SOME

|        |                                      |                               |                                             |                                        |
|--------|--------------------------------------|-------------------------------|---------------------------------------------|----------------------------------------|
|        | S(N and C)                           | S(N or C)                     | Not S(N and C)                              | Not S(N or C)                          |
| In     | In S(N) and<br>In S(N and C)         | In S(N) or<br>In S(C)         | Not In S(N) or<br>Not In S(N and C)         | Not In S(N) and<br>Not In S(C)         |
| Exists | Exists S(N) and<br>Exists S(N and C) | Exists S(N) or<br>Exists S(C) | Not Exists S(N) or<br>Not Exists S(N and C) | Not Exists S(N) and<br>Not Exists S(C) |
| =Some  | =Some S(N) and<br>=Some S(N and C)   | =Some S(N) or<br>=Some S(C)   | Not =Some S(N) or<br>Not =Some S(N and C)   | Not =Some S(N) and<br>Not =Some S(C)   |
| <>Some | <>Some S(N) and<br><>Some S(N and C) | <>Some S(N) or<br><>Some S(C) | Not <>Some S(N) or<br>Not <>Some S(N and C) | Not <>Some S(N) and<br>Not <>Some S(C) |
| <Some  | <Some S(N) and<br><Some S(N and C)   | <Some S(N) or<br><Some S(C)   | Not <Some S(N) or<br>Not <Some S(N and C)   | Not <Some S(N) and<br>Not <Some S(C)   |
| >Some  | >Some S(N) and<br>>Some S(N and C)   | >Some S(N) or<br>>Some S(C)   | Not >Some S(N) or<br>Not >Some S(N and C)   | Not >Some S(N) and<br>Not >Some S(C)   |

Таблица 2: Преобразования запросов с оператором ALL

|       | S(N and C)                            | S(N or C)                          | Not S(N and C)                             | Not S(N or C)                       |
|-------|---------------------------------------|------------------------------------|--------------------------------------------|-------------------------------------|
| =ALL  | =All S(N) or<br>=All S(N and C)       | =All S(N) and<br>=All S(C)         | Not =All S(N) and<br>Not =All S(N and C)   | Not =All S(N) or<br>Not =All S(C)   |
|       | =Some S(N) and<br>=All S(N and C) *   | =Some S(N) and<br>=All S(N or C)   |                                            |                                     |
| <>ALL | <>All S(N) or<br><>All S(N and C)     | <>All S(N) and<br><>All S(C)       | Not <>All S(N) and<br>Not <>All S(N and C) | Not <>All S(N) or<br>Not <>All S(C) |
|       | <>Some S(N) and<br><>All S(N and C) * | <>Some S(N) and<br><>All S(N or C) |                                            |                                     |
| <ALL  | <All S(N) or<br><All S(N and C)       | <All S(N) and<br><All S(C)         | Not <All S(N) and<br>Not <All S(N and C)   | Not <All S(N) or<br>Not <All S(C)   |
|       | <Some S(N) and<br><All S(N and C) *   | <Some S(N) and<br><All S(N or C)   |                                            |                                     |
| >ALL  | >All S(N) or<br>>All S(N and C)       | >All S(N) and<br>>All S(C)         | Not >All S(N) and<br>Not >All S(N and C)   | Not >All S(N) or<br>Not >All S(C)   |
|       | >Some S(N) and<br>>All S(N and C) *   | >Some S(N) and<br>>All S(N or C)   |                                            |                                     |

### 3.2.2. Преобразование с дублированием некоррелированной составляющей

С помощью запроса, отраженного в листинге 6, можно найти все книги, для которых существуют писатели с рейтингом не выше 5 и возрастом не менее 60, прозвище которых совпадает с именем автора соответствующей книги. В данном подзапросе некоррелированная и коррелированная компоненты связаны через AND. При этом его разбиение требует дублирования первой, так как запрос из листинга 7 не эквивалентен исходному. Вместо этого мы можем преобразовать его так, как это показано в листинге 8.

**Листинг 6: Подзапрос после EXISTS: исходный**

```
SELECT B.title
FROM books AS B
WHERE EXISTS (
    SELECT *
    FROM writers AS W
    WHERE W.name = B.author_name AND
          W.age >= 60 AND
          W.rating <= 5
);
```

**Листинг 7: Подзапрос после EXISTS: не эквивалентный**

```
SELECT B.title
FROM books AS B
WHERE EXISTS (
    SELECT *
    FROM writers AS W
    WHERE W.age >= 60 AND
          W.rating <= 5
) AND EXISTS (
    SELECT *
    FROM writers AS W
    WHERE W.name = B.author_name
);
```

## Листинг 8: Подзапрос после EXISTS: преобразованный

```
SELECT T.title
FROM books AS B
WHERE EXISTS (
    SELECT *
    FROM writers AS W
    WHERE W.age >= 60 AND
          W.rating <= 5
) AND EXISTS (
    SELECT *
    FROM writers AS W
    WHERE W.name = B.author_name AND
          W.age >= 60 AND
          W.rating <= 5
);
```

Можно заметить, что некоррелированная часть продублирована во втором подзапросе. Поэтому данное преобразование менее эффективно, чем рассмотренное ранее, поскольку приводит к избыточным вычислениям. Однако оно все равно оказывает влияние на прирост производительности, когда данные распределены специальным образом. Преобразованный запрос быстро вернет False, если не существует писателя не моложе 60 лет с рейтингом не выше 5, поскольку исполнителю запроса не придется проходить по всей таблице writers для каждой строки из books.

### 3.2.3. Преобразование с заменой логической связки

Далее рассмотрим подзапрос из листинга 9. С его помощью можно найти все книги, имеющие рейтинг, больший соответствующих значений среди всех книг, которые либо имеют больше 400 страниц, либо не являются любимыми.

В данном подзапросе некоррелированная и коррелированная компоненты связаны через OR. При этом его разбиение требует замены логической связки на противоположную (OR переходит в AND). Более того, несмотря на кажущуюся корректность, нельзя использовать UNION в



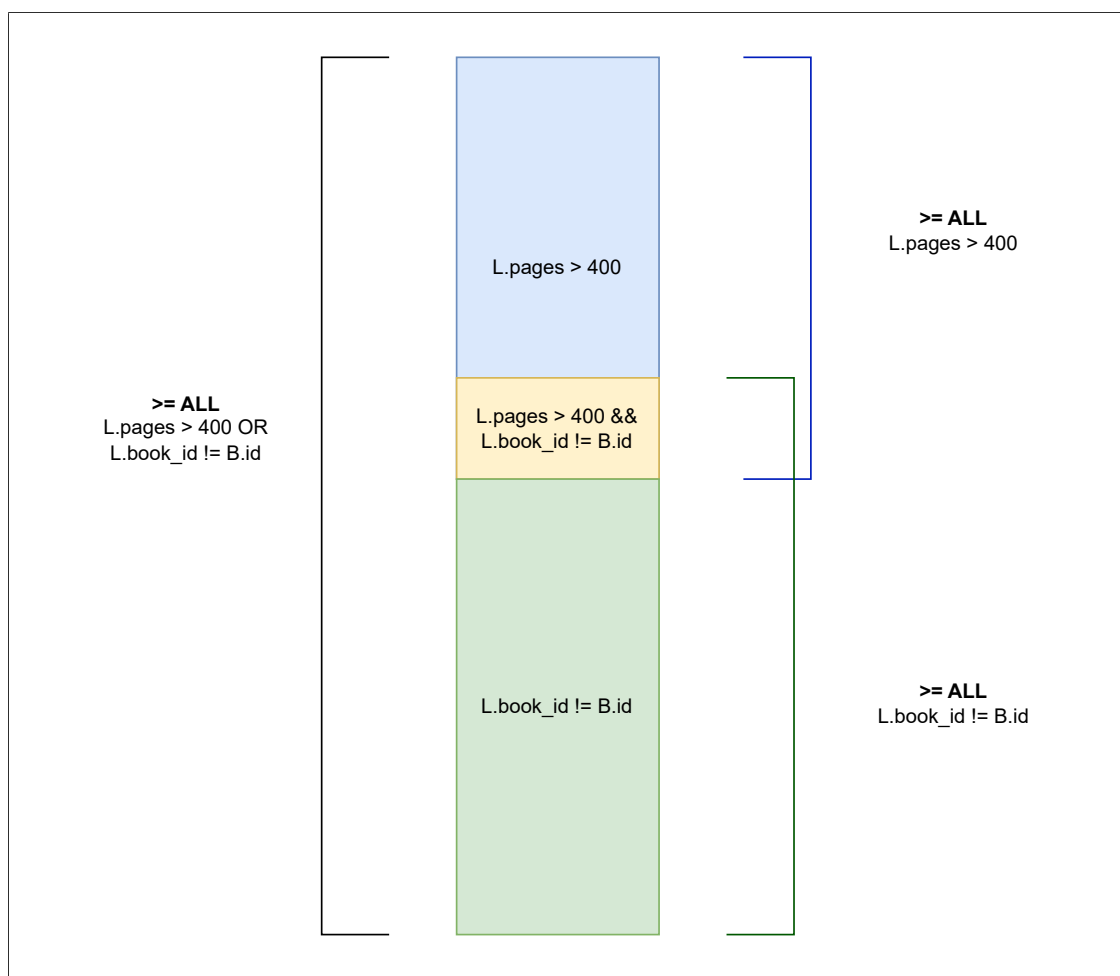


Рис. 1: Неформальная иллюстрация перехода от OR к AND

данном преобразовании. Корректно трансформированный запрос отражен в листинге 10. Неформально данный переход можно описать так: ALL на двух частях одновременно есть ALL по объединению этих частей. Это показано на рисунке 1. А с математической точки зрения это вытекает из свойств минимума и максимума по целочисленному рейтингу.

Предложенное преобразование позволяет исключать книги, рейтинг которых ниже рейтинга хотя бы одной из любимых книг с более чем 400 страницами, без необходимости обработки коррелированной составляющей. Это особенно эффективно при наличии множества высокорейтинговых любимых романов. В качестве иллюстрации рассмотрим таблицы 3 и 4. Оценка логического выражения для всех книг, которые имеют идентификатор, отличный от 4, будет завершена на первой ча-

сти составного условия. Будет получено значение False, на чем вычисление соответствующего результата будет остановлено. Обработка книги с идентификатором 4 потребует дополнительного выполнения коррелированного подзапроса, однако накладные расходы на один такой проход будут несущественными.

### Листинг 9: Подзапрос после ALL: исходный

```
SELECT B.title
FROM books AS B
WHERE B.rating >= ALL (
    SELECT L.rating
    FROM liked_books AS L
    WHERE L.pages > 400 OR
        L.book_id != B.id
);
```

### Листинг 10: Подзапрос после ALL: преобразованный

```
SELECT B.title
FROM books AS B
WHERE B.rating >= ALL (
    SELECT L.rating
    FROM liked_books AS L
    WHERE L.pages > 400
) AND B.rating >= ALL (
    SELECT L.rating
    FROM liked_books AS L
    WHERE L.book_id != B.id
);
```

Таблица 3: books

| id  | rating | pages | title   |
|-----|--------|-------|---------|
| 1   | 7      | 100   | title_1 |
| 2   | 3      | 300   | title_2 |
| 3   | 4      | 100   | title_3 |
| 4   | 10     | 900   | title_4 |
| 5   | 1      | 250   | title_5 |
| ... | ...    | ...   | ...     |
| $n$ | 1      | 250   | title_n |

Таблица 4: liked\_books

| id      | rating | pages | title    |
|---------|--------|-------|----------|
| 4       | 9      | 900   | title_4  |
| $n + 1$ | 10     | 950   | title_n1 |
| $n + 2$ | 10     | 950   | title_n2 |
| ...     | ...    | ...   | ...      |
| $n + k$ | 10     | 950   | title_nk |

### 3.3. Сценарии применения

Следует подчеркнуть, что рассматриваемая оптимизация логических выражений в подзапросах применима только к подзапросам с составным предикатом, содержащим как коррелированную, так и некоррелированную составляющие. Основной запрос при этом должен быть сформирован с помощью операторов ALL, SOME, IN или EXISTS. Пример такого запроса отражен в листинге 11.

**Листинг 11: Подзапрос с составным предикатом**

```
SELECT S.name
FROM students AS S
WHERE S.name IN (
    SELECT T.name
    FROM teachers AS T
    WHERE T.subject = 'Math' OR
           T.age < S.age
);
```

В подобных запросах некоррелированную часть можно выделить всегда, но способ этого выделения зависит от логических связок и от типа оператора сравнения с множеством, что уже было показано ранее.

В случае предиката с AND выполнение преобразованного запроса не требует изменений в коде исполнителя запроса СУБД: достаточно переставить операторы в плане так, чтобы его выполнение шло следующим образом. Сначала среди позиций внутренней таблицы удаляются те, что не удовлетворяют некоррелированной части. Затем для каждой позиции внешней таблицы проверяется уже коррелированная часть, но только по оставшимся после фильтрации позициям. Такой подход позволяет исключить из обработки значительную часть данных, но может потребовать повторных проверок некоррелированного предиката, если того требует соответствующее преобразование. Тем не менее, современные оптимизаторы СУБД уже учитывают подобные простые случаи, поэтому в данном контексте предлагаемая оптимизация не дает дополнительных преимуществ.

Для предиката с OR простая перестановка операторов в плане уже

недостаточна. В этом случае требуется доработать ядро исполнителя запросов: нужен специальный оператор, обеспечивающий корректное выполнение преобразованного запроса. Описание этого оператора приведено в следующем разделе. На примере сценария с OR преимущества рассматриваемой оптимизации особенно наглядны, поэтому основное внимание исследований уделяется именно этому случаю.

Если же предикат включает отрицание, то из-за особенностей поблочной обработки данных и реализации оператора NOT никаких специальных ситуаций не возникает: такие запросы будут работать так же, как и в ранее разобранных сценариях. Соответственно, и выгода от применения рассматриваемой техники окажется равнозначной.

## 4. Реализация

Рассмотрение предложенного метода обработки подзапросов ведется в рамках колоночной СУБД с поддержкой позиционного представления данных и ультра поздней материализации [14]. Вместе эти ключевые особенности обеспечивают возможность применения дополнительных оптимизаций.

### 4.1. Архитектура PosDB

Архитектура PosDB основана на итераторной модели Volcano [12], представляющей собой универсальный и эффективный метод выполнения запросов в СУБД. Более точно, PosDB применяет блочную модификацию этой модели, при которой данные передаются между операторами пакетами, что заметно повышает производительность системы. Кроме того, PosDB — распределенная СУБД, поддерживающая механизмы параллельных вычислений. Такая архитектура обеспечивает горизонтальную масштабируемость, что критично для современных приложений, обрабатывающих в реальном времени большое количество информации.

Для доступа к данным в PosDB применяются отдельные компоненты, именуемые считывателями. Они помогают устранить проблему ромбовидных шаблонов в потоках данных [11, 20], возникающую в сложных иерархиях и приводящую к падению общей эффективности. Сами данные двигаются по дереву операторов, которые делятся на два класса: кортежные и позиционные. Первые передают между узлами полноценные наборы значений, тогда как вторые работают с позициями, указывающими местоположение этих наборов в таблице. Такое разделение повышает эффективность системы за счет сокращения объема передаваемых данных: до определенного этапа, называемого материализацией, между узлами дерева вместо значений двигаются легковесные “указатели” на данные. Для поддержки этого механизма PosDB поддерживает оба типа операторов на уровне плана запроса.

Запросы в PosDB исполняются по сформированным планам — упо-

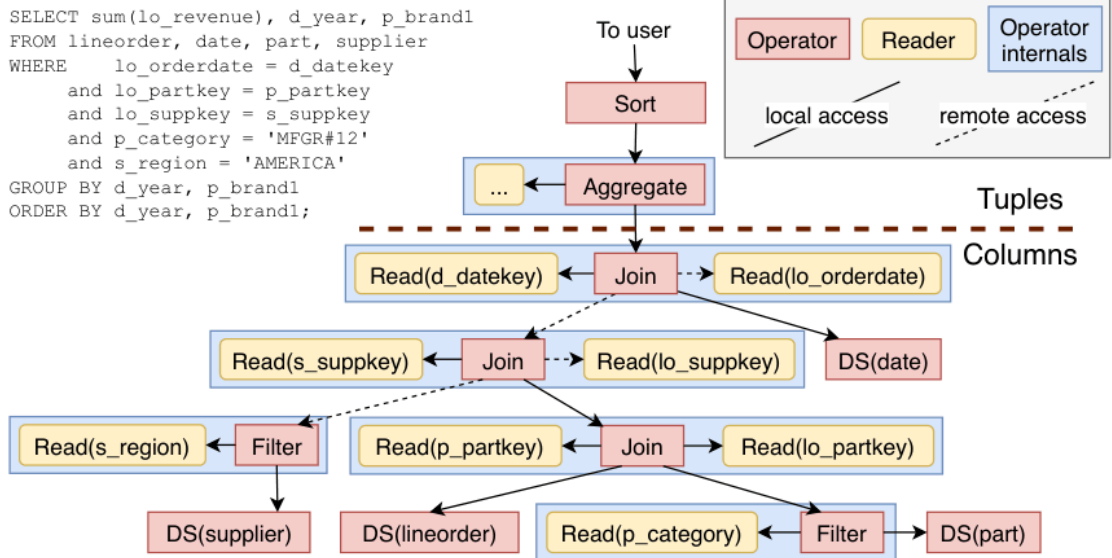


Рис. 2: План запроса в PosDB. Изображение взято из статьи [9].

рядоченным наборам шагов, ведущих к вычислению результата [5]. На первом этапе строится логический план, фиксирующий требуемые узлы. Далее он переводится в физический, где каждому узлу уже сопоставлены конкретные операторы для обработки данных. Такой двухступенчатый подход дает контроль над использованием ресурсов и возможности для различных оптимизаций. Пример возможного плана запроса отражен на рисунке 2.

Таким образом, PosDB — распределенная, колоночная система управления базами данных, использующая позиции и итераторную модель Volcano для обеспечения своей высокой производительности. Ее архитектура обеспечивает масштабируемость и легкость добавления новых операторов, позволяя системе быстро адаптироваться к меняющимся требованиям.

## 4.2. LP оператор

Был разработан оператор Logical Predicate (LP), архитектура которого показана на рисунке 3. Он выполняет координацию обработки запроса, инициируя исполнение различных компонентов плана, и включает в себя следующие элементы:

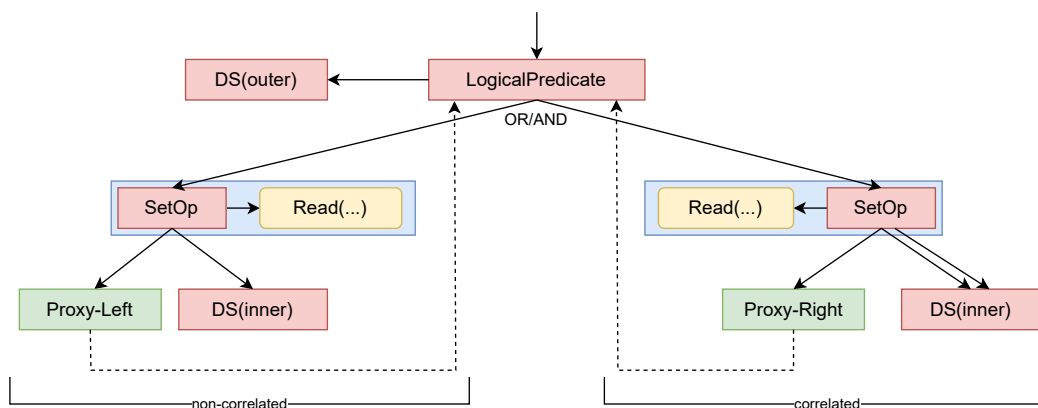


Рис. 3: Архитектура оператора LP

1. Оператор DS (DataSource), предоставляющий данные в виде позиций для внешней таблицы.
2. Предикат, образующий внешний блок WHERE.
3. Два дочерних узла, представляющие части подзапроса.

В PosDB для поддержки выполнения подзапросов введен оператор SetOp. Logical Predicate взаимодействует с двумя экземплярами этого оператора, которые представляют некоррелированную и коррелированную части подзапроса соответственно.

Logical Predicate реализован в двух вариантах: OR-based LP и AND-based LP. Первый предназначен для обработки ситуаций, когда предикат, образующий внешний блок WHERE, представляет собой дизъюнкцию, тогда как второй применяется при наличии конъюнктивного предиката. Внешняя архитектура обеих реализаций сохраняется одинаковой. Однако в их внутренней логике и алгоритмических деталях имеются существенные различия. Ниже представлены описания каждой версии.

Основной функционал OR-based LP распределен по трем ключевым компонентам: Modulator, Inverter и Merger, а также вспомогательным Проxy-оператором, отвечающим за передачу блоков между этими составляющими. Компоненты обмениваются данными, синхронизируют

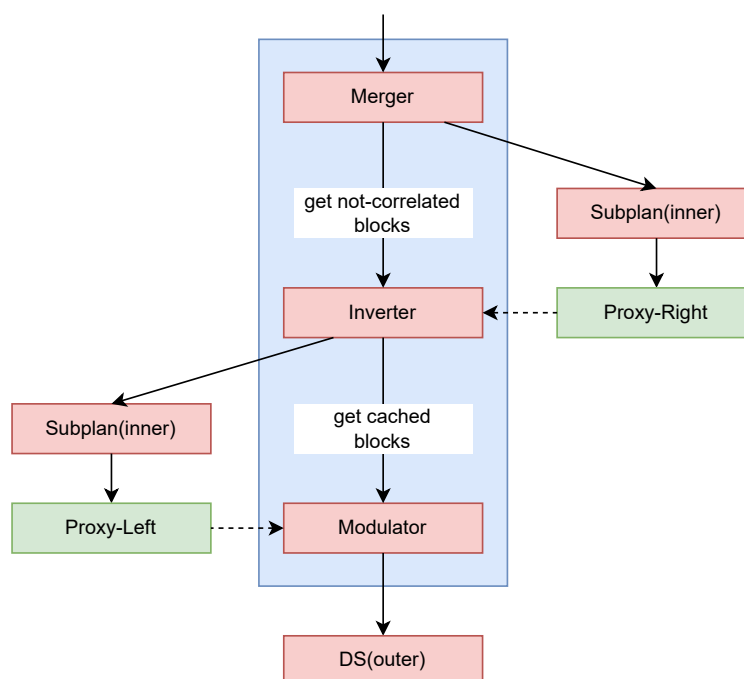


Рис. 4: Поток данных оператора LP

их внешние и внутренние потоки, а также обеспечивают механизмы объединения, исключения и кеширования блоков. Архитектура обработки данных отражена на рисунке 4 и включает в себя следующие этапы:

1. Modulator получает блоки данных от внешнего оператора и, посредством левого Proxy, направляет их в некоррелированный подплан, одновременно осуществляя кеширование этих блоков для последующего обращения.
2. Inverter последовательно читает блоки из некоррелированного подплана до накопления требуемого объема входных данных. Затем он запрашивает у Modulator ранее закешированные блоки. На основе полученных блоков выполняется операция исключения позиций, формирующая набор входных данных, необходимый для выполнения коррелированного подплана.
3. Merger получает результаты как от Inverter, так и от коррелированного подплана. В этом процессе участвует правый Proxy-оператор, который обеспечивает возможность получения внешних отфильтрованных боков соответствующим подпланом. Далее



Merger объединяет коррелированные и некоррелированные выходные данные и выдает окончательный результат LP оператора.

В свою очередь, AND-based Logical Predicate работает по другому и более простому принципу:

1. Сперва идет получение блоков данных от внешнего оператора. Посредством левого Proxy они направляются в некоррелированный подплан.
2. Далее этим подпланом происходит фильтрация полученных позиций. Оставшиеся данные передаются в коррелированный подплан через правый Proxy-оператор.
3. Наконец, результат коррелированного подплана выдается в качестве ответа LP оператора.

Таким образом, предложен новый оператор для обработки подзапросов в колоночном позиционном хранилище данных. Подобные системы позволяют формировать длинные последовательности позиционных операторов, что повышает потенциальную эффективность как применения рассматриваемых оптимизаций, так и выполнения запросов в целом.

### **4.3. Реализация недостающих операторов**

Чтобы обеспечить возможность проведения экспериментов на всех выделенных классах запросов, было необходимо реализовать недостающий оператор для рассматриваемой СУБД. Им оказался оператор NOT. Его архитектура представлена на рисунке 5.

Работа оператора NOT схожа с Logical Predicate и может быть описана следующим образом:

1. Сперва идет получение данных от внешнего оператора.
2. Затем они направляются в дочерний подплан посредством Proxy-оператора.

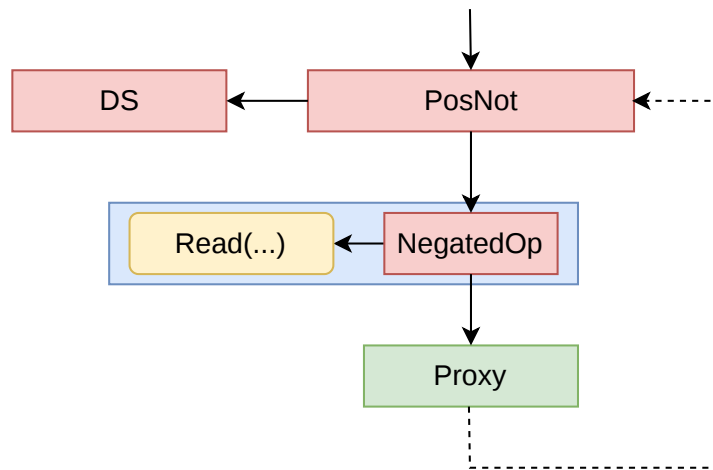


Рис. 5: Архитектура оператора NOT

3. Результат, полученный из этого подплана, выдается в качестве ответа.

В ходе тестирования операторов NOT и LP обнаружились логические ошибки в PosDB, затрагивавшие обработку ряда предикатов. Требовалось внести соответствующие исправления в реализацию ядра. После их применения система стала функционировать корректно, а также открылась возможность проведения экспериментов, описанных в следующем разделе.

## 5. Эксперименты

Для демонстрации эффективности предложенной методики оптимизации подзапросов в рамках системы PosDB был реализован Logical Predicate. Разработка велась на языке C++ 17, что обеспечило хорошую производительность.

### 5.1. Условия экспериментов

Оценка производительности проводилась на ПК со следующим программным и аппаратным обеспечением:

- Программное обеспечение: Debian GNU/Linux 12, Kernel 6.1.0-44-amd64, gcc 12.2.0.
- Аппаратное обеспечение: Intel® Core™ i7-11800H CPU @ 2.30GHz (8 cores), 16GB DDR4 3200MHz RAM, 512GB SSD SAMSUNG MZVL2512HCJQ-00BL2.

В качестве исходных данных были взяты таблицы из набора данных TPC-H с параметром масштабирования, равным 0.2.

### 5.2. Классы запросов, исключенные из рассмотрения

Ранее уже было сказано, что современные оптимизаторы СУБД (включая тот, что используется в PostgreSQL) хорошо оптимизируют составные предикаты с AND. Поэтому предложенная в данной работе оптимизация в этом контексте не дает никаких преимуществ и не рассматривается.

Также было показано, что наличие в запросе отрицания не влечет за собой возникновение специальных ситуаций. Производительность обработки таких запросов и их эквивалентных форм практически одинакова. Например, запрос из листинга 12 покажет примерно те же результаты, что и запрос 13, относящийся к классу 10. Это же касается и запроса с IN, который эквивалентен запросу класса 6 с =SOME.

### Листинг 12: Запрос с NOT ALL: исходный

```
SELECT B.title
FROM books AS B
WHERE NOT B.rating >= ALL (
    SELECT L.rating
    FROM liked_books AS L
    WHERE L.pages > 400 OR
        L.book_id != B.id
);
```

### Листинг 13: Запрос с NOT ALL: эквивалентный

```
SELECT B.title
FROM books AS B
WHERE B.rating < SOME (
    SELECT L.rating
    FROM liked_books AS L
    WHERE L.pages > 400 OR
        L.book_id != B.id
);
```

Рассмотрение запроса класса 8 тоже не имеет смысла, так как на неконстантных данных  $\langle \rangle$ SOME всегда будет выдавать либо все множество, либо его же без одного элемента. Более того, соответствующее сравнение будет обрабатываться очень быстро. Из-за этого в подобных запросах (пример в листинге 14) нет места для оптимизации. Это же касается и альтернативного преобразования запросов класса 16, поскольку оно подразумевает использование  $\langle \rangle$ SOME.

### Листинг 14: Запрос Q3 (класс 8)

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey <> SOME (
    SELECT l_orderkey FROM lineitem
    WHERE l_suppkey > o_orderkey OR l_suppkey != 39
);
```

В ходе анализа запроса класса 16 было выяснено, что на достаточно больших для сравнения данных обработка  $\langle \rangle$ ALL занимает слишком много времени. Это касается как PostgreSQL, так и PosDB с LP. То есть

применение предложенного метода на запросах, подобных 15, не имеет смысла.

#### Листинг 15: Запрос Q8 (класс C16)

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey <> ALL (
    SELECT l_orderkey FROM lineitem
    WHERE l_orderkey = o_orderkey OR l_suppkey != 451
);
```

### 5.3. Проведенные эксперименты

В результате для проведения экспериментов были отобраны следующие классы: C4, C6, C10, C12, C14, C18, C20. Соответствующие им запросы, а также их преобразованные формы, приведены в листингах 16–35.

#### Листинг 16: Запрос Q1 (класс C4)

```
SELECT COUNT(o_orderkey) FROM orders
WHERE EXISTS(
    SELECT l_extendedprice FROM lineitem
    WHERE o_orderkey > l_orderkey OR
        3 * l_extendedprice * l_discount * l_tax > 2314
);
```

#### Листинг 17: Запрос Q1' (класс C4')

```
SELECT COUNT(o_orderkey) FROM orders
WHERE EXISTS (
    SELECT l_extendedprice FROM lineitem
    WHERE 3 * l_extendedprice * l_discount * l_tax > 2314
) OR EXISTS (
    SELECT l_extendedprice FROM lineitem
    WHERE o_orderkey > l_orderkey
);
```

### Листинг 18: Запрос Q2 (класс C6)

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey = SOME (
    SELECT l_orderkey FROM lineitem
    WHERE l_suppkey > o_orderkey OR l_suppkey > 39
);
```

### Листинг 19: Запрос Q2' (класс C6')

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey = SOME (
    SELECT l_orderkey FROM lineitem
    WHERE l_suppkey > 39
) OR o_orderkey = SOME (
    SELECT l_orderkey FROM lineitem
    WHERE l_suppkey > o_orderkey
);
```

### Листинг 20: Запрос Q4 (класс C10)

```
SELECT COUNT(p_partkey) FROM part
WHERE p_retailprice < SOME (
    SELECT 3 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

### Листинг 21: Запрос Q4' (класс C10')

```
SELECT COUNT(p_partkey) FROM part
WHERE p_retailprice < SOME (
    SELECT 3 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE l_suppkey = 451
) OR p_retailprice < SOME (
    SELECT 3 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity
);
```

### Листинг 22: Запрос Q5 (класс C12)

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice > SOME (
    SELECT 914 + 3 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

### Листинг 23: Запрос Q5' (класс C12')

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice > SOME (
    SELECT 914 + 3 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE l_suppkey = 451
) OR p_retailprice > SOME (
    SELECT 914 + 3 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity
);
```

### Листинг 24: Запрос Q6 (класс C14)

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey = ALL (
    SELECT l_orderkey FROM lineitem
    WHERE o_orderkey = l_orderkey OR l_suppkey = 451
);
```

### Листинг 25: Запрос Q6' (класс C14')

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey = ALL (
    SELECT l_orderkey FROM lineitem
    WHERE l_suppkey = 451
) AND o_orderkey = ALL (
    SELECT l_orderkey FROM lineitem
    WHERE o_orderkey = l_orderkey
);
```

### Листинг 26: Запрос Q7 (класс C14)

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey = ALL (
    SELECT l_orderkey FROM lineitem
    WHERE o_orderkey = l_orderkey OR l_suppkey = 451
);
```

### Листинг 27: Запрос Q7' (класс C14')

```
SELECT COUNT(o_orderkey) FROM orders
WHERE o_orderkey = SOME (
    SELECT l_orderkey FROM lineitem
    WHERE l_suppkey = 451
) AND o_orderkey = ALL (
    SELECT l_orderkey FROM lineitem
    WHERE o_orderkey = l_orderkey OR l_suppkey = 451
);
```

### Листинг 28: Запрос Q10 (класс C18)

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice < ALL (
    SELECT 930 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

### Листинг 29: Запрос Q10' (класс C18')

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice < ALL (
    SELECT 930 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE l_suppkey = 451)
AND p_retailprice < ALL (
    SELECT 930 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity
);
```



### Листинг 30: Запрос Q11 (класс C18)

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice < ALL (
    SELECT 290 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

### Листинг 31: Запрос Q11' (класс C18)

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice < SOME (
    SELECT 290 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE l_suppkey = 451)
AND p_retailprice < ALL (
    SELECT 290 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

### Листинг 32: Запрос Q12 (класс C20)

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice > ALL (
    SELECT 2.95 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

### Листинг 33: Запрос Q12' (класс C20')

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice > ALL (
    SELECT 2.95 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE l_suppkey = 451
) AND p_retailprice > ALL (
    SELECT 2.95 * l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity
);
```

Таблица 5: Сравнение производительности по классам

| Класс | Запрос | PG original<br>(сек.) | PG rewritten<br>(сек.) | PosDB rewritten<br>(сек.) | Улучшение<br>PG-orig /<br>PosDB-rewr | Улучшение<br>PG-rewr /<br>PosDB-rewr |
|-------|--------|-----------------------|------------------------|---------------------------|--------------------------------------|--------------------------------------|
| C4    | Q1     | $\infty$              | 0.33                   | 0.16                      | $\infty$                             | 1.5x                                 |
| C6    | Q2     | 13089.00              | 7830.00                | 12.14                     | 1078.17x                             | 644.98x                              |
| C10   | Q4     | 1077.00               | 3.02                   | 0.46                      | 2341.30x                             | 6.57x                                |
| C12   | Q5     | 14.55                 | 13.95                  | 1.92                      | 7.58x                                | 7.27x                                |
| C14   | Q6     | 11.53                 | $\infty$               | 0.04                      | 288.25x                              | $\infty$                             |
| C14   | Q7     | 2.03                  | 50.71                  | 15.72                     | 0.13x                                | 3.23x                                |
| C18   | Q10    | 63.50                 | 61.23                  | 9.13                      | 6.96x                                | 6.71x                                |
| C18   | Q11    | 0.32                  | 0.36                   | 11.15                     | 0.03x                                | 0.03x                                |
| C20   | Q12    | 1514.00               | 92.02                  | 15.96                     | 94.86x                               | 5.77x                                |
| C20   | Q13    | 0.31                  | 0.35                   | 15.42                     | 0.02x                                | 0.02x                                |

### Листинг 34: Запрос Q13 (класс C20)

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice > ALL (
    SELECT 1900 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

### Листинг 35: Запрос Q13' (класс C20')

```
SELECT COUNT(p_retailprice) FROM part
WHERE p_retailprice > SOME (
    SELECT 1900 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE l_suppkey = 451
) AND p_retailprice > ALL (
    SELECT 1900 + l_extendedprice * l_discount * l_tax FROM lineitem
    WHERE p_size = l_quantity OR l_suppkey = 451
);
```

Результаты экспериментов приведены в таблице 5. Можно заменить, что применение предложенного метода оптимизации выполнения подзапросов оказывается эффективным на всех рассматриваемых классах запросов. В особенной степени это наблюдается для классов C6 и C10. При этом использование альтернативных преобразований оказывается неэффективным.

Также было выполнено исследование изменения времени выполнения запроса при варьировании селективности его некоррелированного

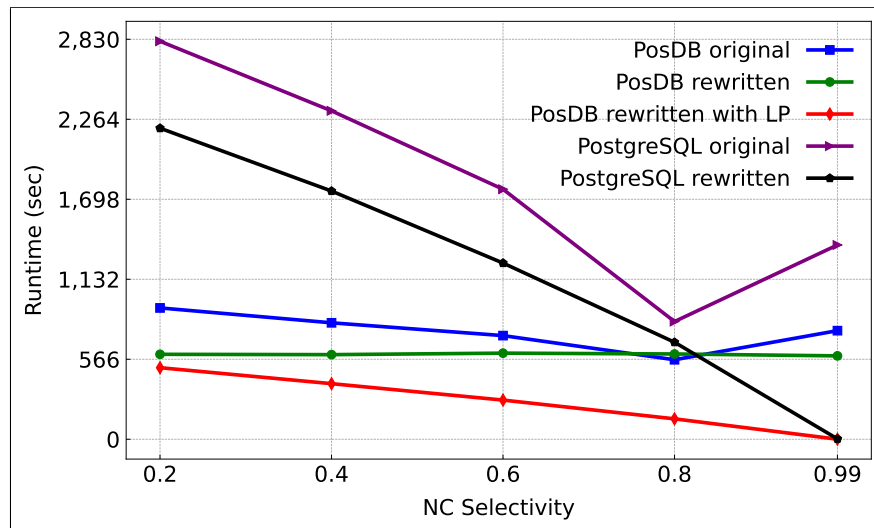


Рис. 6: Время выполнения в зависимости от селективности NC

подзапроса. Для упрощения параметризации в запрос введена переменная  $X$ : значения 39, 1198, 734, 210 и 451 обеспечивают селективность некоррелированного подзапроса в 20%, 40%, 60%, 80% и 99.9% соответственно. При этом селективность коррелированного фиксирована на уровне 40%. Было проведено измерение времени выполнения запросов для каждого заданного уровня селективности в пяти различных конфигурациях:

1. Исходный запрос в PosDB.
2. Преобразованный запрос в PosDB без LP.
3. Преобразованный запрос в PosDB с LP.
4. Исходный запрос в PostgreSQL.
5. Преобразованный запрос в PostgreSQL.

Результаты данного исследования представлены на рисунке 6. Можно заметить, что время выполнения запроса с любым значением селективности в PosDB без LP остается примерно на одном уровне. Это связано с использованием жестко заданного плана выполнения запроса и отсутствием в PosDB механизмов его корректировки во время выполнения. Также видно, что время выполнения преобразованного запроса

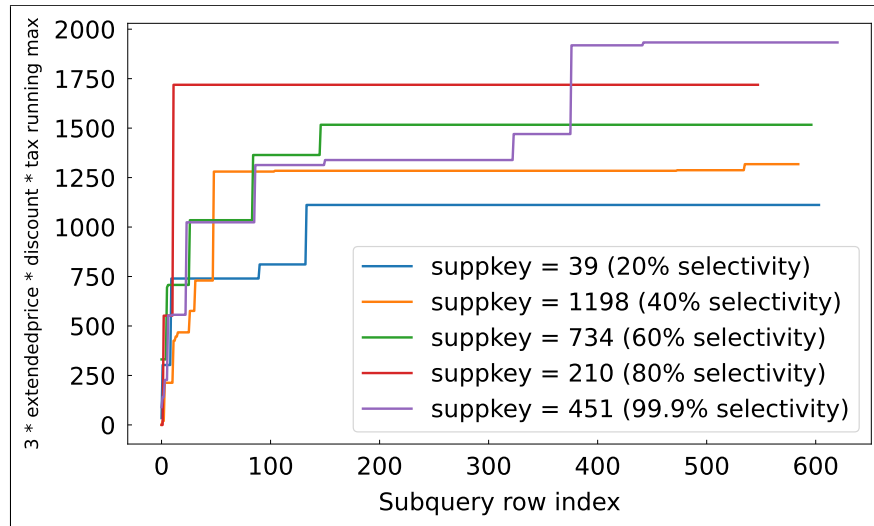


Рис. 7: Максимум значения выражения на разных этапах выполнения запроса

в PosDB с LP и PostgreSQL уменьшается линейно при увеличении селективности некоррелированного подзапроса. При этом оно стремится к нулю при увеличении этой селективности практически до 100%.

Следует отметить, что PostgreSQL выполняет любой исходный и соответствующий ему преобразованный запрос за разное время. Это свидетельствует о том, что данная СУБД не применяет предложенное в данной работе преобразование запроса. Особый интерес представляет селективность подзапроса, равная 99.9%. PostgreSQL обрабатывает соответствующий запрос примерно за то же время, что и аналогичный с 60% селективностью. То есть значительно дольше, чем в случае 80% селективности некоррелированного подзапроса. Это объясняется тем, что PostgreSQL применяет оптимизацию прекращения вычислений на ранней стадии при оценке конструкции запроса, однако она оказывается неэффективной в данном случае.

На рисунке 7 отражен максимум вычисленного значения выражения “ $3 * extendedprice * discount * tax$ ” на определенном этапе для запросов, соответствующих рассматриваемым значениям селективности. Для 99.9% селективности некоррелированного подзапроса максимальное значение этого выражения достигается лишь на середине сканирования. Принимая во внимание, что в таблице PART столбец retailprice имеет рав-

номерное распределение в диапазоне [900; 1900], это свидетельствует о следующем. Исполнитель запроса не сможет произвести прекращение вычислений на ранней стадии для большей части совпадающих строк. То есть рассматриваемое значение селективности оказывается неудачным для этой оптимизации. Предложенный же в данной работе метод обработки подзапросов не имеет такой уязвимости.

Таким образом, эксперименты явно продемонстрировали, что на многих классах запросов применение предложенного метода оптимизации обеспечивает многократное увеличение производительности.

# Заключение

Был разработан метод оптимизации выполнения запросов в колоночной позиционной СУБД, основанный на выделении из составных подзапросов некоррелированных частей. Как результат, были выполнены следующие задачи:

1. Произведено знакомство с предметной областью обработки коррелированных подзапросов. Написан обзор существующих методов оптимизации их выполнения.
2. Выделены классы запросов, на которых может быть применен метод оптимизации выполнения подзапросов, основанный на выделении из них некоррелированных частей. Определены те из них, где его применение обеспечивает увеличение производительности.
3. Разработан метод оптимизации выполнения подзапросов. Он успешно внедрен в архитектуру позиционной колоночной СУБД PosDB.
4. Реализованы недостающие операторы для рассматриваемой СУБД и обеспечена возможность проведения экспериментов на всех выделенных классах запросов.
5. Осуществлен сравнительный анализ производительности системы до и после внедрения предложенного метода оптимизации. Получено многократное итоговое ускорение обработки запросов. При определенных условиях оно достигло 600-кратного.

Предложенные подходы опубликованы в статье “Speeding up SQL subqueries via decoupling of non-correlated predicate” [22], представленной на 14-ой международной конференции по моделированию и инженерии данных MEDI, проводившейся в Египте (г. Каир). Ее работы публикуются Springer и индексируются в Scopus.

## Список литературы

- [1] Abadi Daniel, Boncz Peter, Harizopoulos Stavros. The Design and Implementation of Modern Column-Oriented Database Systems.— Hanover, MA, USA : Now Publishers Inc., 2013. — ISBN: [1601987544](#), [9781601987549](#).
- [2] Abadi Daniel J., Boncz Peter A., Harizopoulos Stavros. Column-oriented database systems // [Proc. VLDB Endow.](#) — 2009. — Aug.. — Vol. 2, no. 2. — P. 1664–1665. — URL: <https://doi.org/10.14778/1687553.1687625>.
- [3] Bellamy Michael. Pros and Cons of CTEs and Subqueries in SQL. — 2026. — URL: <https://www.thedataschool.co.uk/michael-bellamy/pros-and-cons-of-ctes-and-subqueries-in-sql> (дата обращения: 30.04.2026).
- [4] Bruno Nicolas, Galindo-Legaria César, Joshi Milind. [Query Decorrelation in the Fabric Data Warehouse](#) // Companion of the 2025 International Conference on Management of Data. — SIGMOD/PODS '25. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 297–309. — URL: <https://doi.org/10.1145/3722212.3724448>.
- [5] A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store / George A. Chernishev, Viacheslav Galaktionov, Valentin V. Grigorev et al. // Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP) co-located with the 25th International Conference on Extending Database Technology and the 25th International Conference on Database Theory (EDBT/ICDT 2022), Edinburgh, UK, March 29, 2022 / Ed. by Kostas Stefanidis, Lukasz Golab. — Vol. 3130 of CEUR Workshop Proceedings. — CEUR-WS.org, 2022. — P. 21–30. — URL: <http://ceur-ws.org/Vol-3130/paper3.pdf>.

- [6] Elmasri Ramez, Navathe Shamkant B. Fundamentals of Database Systems, 3rd Edition. — Addison-Wesley-Longman, 2000. — ISBN: [978-0-8053-1755-8](#).
- [7] Enhanced subquery optimizations in Oracle / Srikanth Bellamkonda, Rafi Ahmed, Andrew Witkowski et al. // [Proc. VLDB Endow.](#) — 2009. — Aug.. — Vol. 2, no. 2. — P. 1366–1377. — URL: <https://doi.org/10.14778/1687553.1687563>.
- [8] [Execution strategies for SQL subqueries](#) / Mostafa Elhemali, César A. Galindo-Legaria, Torsten Grabs, Milind M. Joshi // Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data. — SIGMOD '07. — New York, NY, USA : Association for Computing Machinery, 2007. — P. 993–1004. — URL: <https://doi.org/10.1145/1247480.1247598>.
- [9] Galaktionov Viacheslav, Klyuchikov Evgeniy, Chernishev George A. Position Caching in a Column-Store with Late Materialization: An Initial Study // Proceedings of the 22nd International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data co-located with EDBT/ICDT 2020 Joint Conference, DOLAP@EDBT/ICDT 2020, Copenhagen, Denmark, March 30, 2020 / Ed. by Il-Yeol Song, Katja Hose, Oscar Romero. — Vol. 2572 of CEUR Workshop Proceedings. — CEUR-WS.org, 2020. — P. 89–93. — URL: <http://ceur-ws.org/Vol-2572/short14.pdf>.
- [10] GeeksForGeeks. SQL Join vs Subquery. — 2025. — URL: <https://www.geeksforgeeks.org/sql/sql-join-vs-subquery> (дата обращения: 30.04.2026).
- [11] Georgii Chernyshev. Models and Methods for Query Planning and Execution in a Column-Oriented DBMS with Late Materialization Support. — Dissertation submitted for a Degree of Candidate of Physical and Mathematical Sciences. — 2025. — St. Petersburg State University. URL:



<https://disser.spbu.ru/zashchita-uchenoj-stepeni-spbgu/1503-chernyshev-georgii-alekseevich.html>.

- [12] Graefe G. Volcano — An Extensible and Parallel Query Evaluation System // [IEEE Trans. on Knowl. and Data Eng.](#) — 1994. — Feb.. — Vol. 6, no. 1. — P. 120–135. — URL: <https://doi.org/10.1109/69.273032>.
- [13] Harizopoulos Stavros, Abadi Daniel, Boncz Peter. Column-Oriented Database Systems, VLDB 2009 Tutorial. — 2009. — URL: [nms.csail.mit.edu/~stavros/pubs/tutorial2009-column\\_stores.pdf](https://nms.csail.mit.edu/~stavros/pubs/tutorial2009-column_stores.pdf).
- [14] [Hybrid Materialization in a Disk-Based Column-Store](#) / Evgeniy Klyuchikov, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 7th Joint International Conference on Data Science & Management of Data (11th ACM IKDD CODS and 29th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2024. — P. 164–172. — URL: <https://doi.org/10.1145/3632410.3632422>.
- [15] Kandl Susanne, Chandrashekar Sandeep. [Reasonability of MC/DC for safety-relevant software implemented in programming languages with short-circuit evaluation](#) // 16th IEEE International Symposium on Object/component/service-oriented Real-time distributed Computing (ISORC 2013). — 2013. — P. 1–6.
- [16] Kastrati Fisnik, Moerkotte Guido. [Generating Optimal Plans for Boolean Expressions](#) // 2018 IEEE 34th International Conference on Data Engineering (ICDE). — 2018. — P. 1013–1024.
- [17] Kim Albert, Madden Samuel. Optimizing Disjunctive Queries with Tagged Execution // [Proc. ACM Manag. Data.](#) — 2024. — May. — Vol. 2, no. 3. — 25 p. — URL: <https://doi.org/10.1145/3654961>.
- [18] Neumann Thomas, Leis Viktor, Kemper Alfons. The Complete Story of Joins (in HyPer) // Datenbanksysteme für Business, Technologie

- und Web (BTW 2017), 17. Fachtagung des GI-Fachbereichs „Datenbanken und Informationssysteme“ (DBIS), 6.-10. März 2017, Stuttgart, Germany, Proceedings / Ed. by Bernhard Mitschang, Daniela Nicklas, Frank Leymann et al. — LNI. — GI, 2017. — P. 31–50. — URL: <https://dl.gi.de/handle/20.500.12116/657>.
- [19] [PosDB: A Distributed Column-Store Engine](#) / George A. Chernishev, Viacheslav Galaktionov, Valentin D. Grigorev et al. // Perspectives of System Informatics - 11th International Andrei P. Ershov Informatics Conference, PSI 2017, Moscow, Russia, June 27-29, 2017, Revised Selected Papers / Ed. by Alexander K. Petrenko, Andrei Voronkov. — Vol. 10742 of Lecture Notes in Computer Science. — Springer, 2017. — P. 88–94. — URL: [https://doi.org/10.1007/978-3-319-74313-4\\_7](https://doi.org/10.1007/978-3-319-74313-4_7).
- [20] [PosDB: An Architecture Overview](#) / George A. Chernishev, Vyacheslav Galaktionov, Valentin D. Grigorev et al. // [Program. Comput. Softw.](#) — 2018. — Vol. 44, no. 1. — P. 62–74. — URL: <https://doi.org/10.1134/S0361768818010024>.
- [21] Silberschatz Abraham, Korth Henry, Sudarshan S. Database Systems Concepts. — 5 edition. — USA : McGraw-Hill, Inc., 2005. — ISBN: [0072958863](#).
- [22] [Speeding up SQL subqueries via decoupling of non-correlated predicate](#) / Dmitrii Radivonchik, Yakov Kuzin, Anton Chizhov et al. // 14th International Conference on Model and Data Engineering (MEDI 2025). — 2025. — Paper accepted.
- [23] TPC BENCHMARK(TM) H (Decision Support) Standard Specification Revision 3.0.1. — [https://www.tpc.org/TPC\\_Documents\\_Current\\_Versions/pdf/TPC-H\\_v3.0.1.pdf](https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H_v3.0.1.pdf).
- [24] Zhao Junyi, Zhang Huanchen, Gao Yihan. Efficient Query Reoptimization with Judicious Subquery Selections // [Proc. ACM](#)

Manag. Data. — 2023. — Jun.. — Vol. 1, no. 2. — 26 p. — URL:  
<https://doi.org/10.1145/3589330>.

- [25] zHyperLink: Low-latency I/O for Db2 on IBM Z and DS8880 storage / D. Craddock, A. Hoshikawa, J. Josten et al. // [IBM Journal of Research and Development](#). — 2018. — Vol. 62, no. 2/3. — P. 13:1–13:10.